

3.1 Regresión lineal

M. Sc. Huáscar Fedor Gonzales Guzmán · Carrera de Ingeniería Informática · UATF · Gestión 2026-02

La regresión lineal es el primer algoritmo de aprendizaje supervisado que estudiamos y el punto de partida de casi todo el machine learning. Resuelve un problema de **regresión**: predecir un valor continuo y —un precio, una temperatura, una calificación— a partir de una o varias variables de entrada x . Su hipótesis de fondo es que la relación entre las entradas y la salida es *aproximadamente lineal*, y el algoritmo busca la recta —o el hiperplano— que mejor atraviesa la nube de datos. **Aprender**, aquí, significa hallar los parámetros que hacen que las predicciones se parezcan lo más posible a los datos reales.

3.1.1 Modelo, función de costo y descenso de gradiente

Todo modelo de regresión lineal se define por tres piezas: el **modelo** (qué forma tiene la predicción), la **función de costo** (cómo medimos el error) y el **método de optimización** (cómo hallamos los mejores parámetros). Veamos las tres.

a) El modelo (la hipótesis)

Para una sola variable de entrada —regresión lineal simple— el modelo es una recta:

$$\hat{y} = w \cdot x + b$$

donde $\hat{y}$ es el valor predicho, w es el **peso** o pendiente (cuánto cambia $\hat{y}$ por cada unidad de x) y b es el **sesgo** o intercepto (dónde corta la recta al eje vertical).

Cuando hay varias variables de entrada —regresión lineal múltiple— cada una tiene su propio peso y el modelo pasa a ser un hiperplano:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^T x + b$$

NOTA · **Lineal en los parámetros, no en los datos**

El modelo es lineal en los parámetros w y b , no necesariamente en los datos. Si construimos una nueva variable $x_2 = x_1^2$, seguimos usando exactamente el mismo algoritmo y obtenemos una regresión polinomial (tema 3.2). Esta idea sorprende y conviene subrayarla.

b) Los parámetros

Lo único que el algoritmo puede ajustar son los pesos w y el sesgo b . Las entradas x y las salidas y están fijas: son los datos. Entrenar el modelo significa buscar los valores de w y b que minimicen el error.

c) La función de costo (MSE)

Necesitamos una medida numérica de qué tan mal predice el modelo. La estándar en regresión es el Error Cuadrático Medio (MSE):

DEFINICIÓN · Error Cuadrático Medio (MSE)

Promedio de los cuadrados de las diferencias entre lo predicho y lo real. Es siempre positivo y, cuanto menor sea, mejor es el ajuste del modelo a los datos.

$$J(w, b) = (1 / 2m) \cdot \sum_i (\hat{y}^{(i)} - y^{(i)})^2$$

con m el número de ejemplos. Tres detalles que conviene entender bien:

Por qué al cuadrado. Penaliza más los errores grandes, elimina el signo (los errores positivos y negativos no se cancelan) y produce una función **convexa**: con un único mínimo global y sin mínimos locales donde quedar atrapado.

El factor $1/2$. Es puramente cosmético. Al derivar, el exponente 2 baja y se cancela con el $1/2$, dejando fórmulas limpias. No cambia dónde está el mínimo.

La forma de J . Vista como función de w y b , es un paraboloide —un tazón—. Minimizar el costo equivale a encontrar el fondo del tazón.

d) Descenso de gradiente

¿Cómo llegamos al fondo del tazón? El **descenso de gradiente** parte de valores cualesquiera de w y b y baja paso a paso siguiendo la pendiente. Necesitamos las derivadas parciales del costo (aquí se ve por qué el $1/2$ era conveniente):

$$\begin{aligned}\partial J / \partial w &= (1/m) \cdot \sum_i (\hat{y}^{(i)} - y^{(i)}) \cdot x^{(i)} \\ \partial J / \partial b &= (1/m) \cdot \sum_i (\hat{y}^{(i)} - y^{(i)})\end{aligned}$$

y aplicamos repetidamente la regla de actualización:

$$w := w - \alpha \cdot (\partial J / \partial w) \qquad b := b - \alpha \cdot (\partial J / \partial b)$$

α es la **tasa de aprendizaje** (learning rate). El signo negativo es la esencia del método: nos movemos en dirección *contraria* al gradiente, que es la de mayor descenso.

ADVERTENCIA · Dos errores frecuentes

1) La tasa de aprendizaje. Si α es demasiado grande, el algoritmo oscila o diverge —se sale del tazón—; si es demasiado pequeña, converge lentísimo.

2) Actualización simultánea. w y b deben actualizarse a la vez: calcula ambos gradientes con los valores viejos y solo entonces actualiza los dos.

NOTA · Alternativa exacta: la ecuación normal

Como J es convexa, su mínimo está donde las derivadas valen cero. Resolviendo ese sistema se obtiene una fórmula cerrada, $\theta = (X^T X)^{-1} X^T y$, sin α ni iteraciones. Es exacta, pero cuesta $O(n^3)$ e inestable con colinealidad; el descenso de gradiente escala mejor y es el que conecta con el deep learning.

e) Ejemplo resuelto a mano

EJEMPLO · Ajustar una recta a cinco puntos

Nada fija mejor estas ideas que calcular con números. Usaremos la solución cerrada del caso simple sobre el siguiente conjunto de datos.

Conjunto de datos

x	1	2	3	4	5
y	2	4	5	4	5

Fórmulas de la solución cerrada

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \quad b = \bar{y} - w \cdot \bar{x}$$

Medias: $\bar{x} = (1+2+3+4+5)/5 = 3$ y $\bar{y} = (2+4+5+4+5)/5 = 4$.

Numerador: $(-2)(-2)+(-1)(0)+(0)(1)+(1)(0)+(2)(1) = 6$.

Denominador: $4+1+0+1+4 = 10$.

Por tanto $w = 6/10 = 0.6$ y $b = 4 - 0.6 \cdot 3 = 2.2$. La recta ajustada es $\hat{y} = 0.6 \cdot x + 2.2$.

Predicciones y residuos

x	y real	$\hat{y}$ predicho	residuo ($\hat{y} - y$)
1	2	2.8	+0.8
2	4	3.4	-0.6
3	5	4.0	-1.0
4	4	4.6	+0.6
5	5	5.2	+0.2

Suma de errores al cuadrado: $SSE = 0.64+0.36+1.0+0.36+0.04 = 2.4$, de modo que $MSE = 2.4/5 = 0.48$.

f) Una iteración de descenso de gradiente

Para ver la mecánica del método iterativo, arranquemos con $w = 0$, $b = 0$, $\alpha = 0.01$. Todas las predicciones iniciales son 0, así que los errores ($\hat{y} - y$) son -2, -4, -5, -4, -5.

$$\begin{aligned}\partial J / \partial w &= (1/5)[(-2)(1)+(-4)(2)+(-5)(3)+(-4)(4)+(-5)(5)] = -66/5 = -13.2 \\ \partial J / \partial b &= (1/5)(-2-4-5-4-5) = -20/5 = -4\end{aligned}$$

$$w := 0 - 0.01 \cdot (-13.2) = 0.132 \quad b := 0 - 0.01 \cdot (-4) = 0.04$$

Tras un solo paso ya nos movemos hacia **(0.6, 2.2)**; repitiendo cientos de iteraciones convergemos a la misma solución cerrada. **Eso es el aprendizaje.**

g) Verificación en Python

Comprobemos ambos caminos —la solución cerrada de scikit-learn y el descenso de gradiente desde cero— sobre el mismo conjunto de datos.

In [1]:

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 5, 4, 5])

modelo = LinearRegression().fit(X, y)
print('w =', round(modelo.coef_[0], 3))
print('b =', round(modelo.intercept_, 3))
```

Out [1]:

```
w = 0.6
b = 2.2
```

In [2]:

```
# Descenso de gradiente implementado desde cero
w, b, alpha, m = 0.0, 0.0, 0.01, len(y)
x = X.ravel()

for _ in range(10_000):
    y_pred = w * x + b
    error = y_pred - y
    dw = (error * x).mean() # dJ/dw
    db = error.mean()       # dJ/db
    w -= alpha * dw         # actualizacion
    b -= alpha * db         # simultanea

print(f'w = {w:.3f}  b = {b:.3f}')
```

Out [2]:

```
w = 0.600  b = 2.200
```

Los dos métodos llegan al mismo par **(w, b) = (0.6, 2.2)**. La ecuación normal lo obtiene de golpe; el descenso de gradiente lo alcanza paso a paso —y es esta segunda idea la que reaparecerá en todo lo que sigue del curso.

NOTA • Síntesis del tema

Un modelo de regresión lineal son tres piezas: **modelo** (recta o hiperplano), **costo** (MSE, un tazón convexo) y **optimización** (ecuación normal exacta, o descenso de gradiente iterativo). Las tres reaparecen —con costos y modelos distintos— en casi todos los algoritmos del resto de la materia.